



PROGRAMMING FUNDAMENTALS

CS – 101

Instructor: Maria N. Samad

September 14th, 2022

September 16th, 2022

FUNCTIONS

- A named sequence of statements that performs a computation
- In simpler terms, a function “takes in” an argument value, and “return” a result
- The result is called the ***return value***

FUNCTION DEFINITION VS FUNCTION CALL

- **Function Definition:**

- Specifying the name of the function and writing the sequence of statements

- Syntax:

```
def function_name(argument_list):  
    statement_1  
    statement_2  
    ...  
    statement_n
```

- **Function Calling:**

- Invoking or “calling” a function by its name during program execution

- Syntax:

```
(return_value) = function_name(specific_arguments)
```

- **return_value** may not always be required

WHY FUNCTIONS?

- Makes a program easier to read and declutter
- Can define repetitive code once in function, and then just call it multiple times
- Need to update in one place instead of doing it in multiple places if being repeated
- Divides the program into smaller portions, so easier to debug
- A function defined in one program can be used for other programs

CATEGORIES OF FUNCTIONS

1. Built-in Functions:

- Predefined by the programming language
- Cannot be modified as per user's needs
- Performs many basic operations for the user

2. User-defined Functions:

- Defined by the programmer
- Can be written to perform any of the user-defined tasks
- Gives a program more versatility

BUILT-IN FUNCTIONS

- The two most common categories of built-in functions are as follows:
 1. Type Conversion functions
 2. Math functions

TYPE CONVERSION FUNCTIONS

- Converts values from one type to another
 - For e.g., **int** function takes any input and converts to an integer, as follows:
 - `int('32')` → output = ?
 - `int (32.309)` → output = ?
 - `int("Hello")` → output = ?
 - **float** function converts integers and strings to Floating point number if possible
 - `float('32')` → output = ?
 - `float (32.309)` → output = ?
 - `float("Hello")` → output = ?
 - **str** function converts its argument into String
 - `str(32)` → output = ?
 - `str (32.309)` → output = ?
 - `str("Hello")` → output = ?

MATH FUNCTIONS

- Python has a separate ***math module*** that provides some common mathematic operations
- **Module**
 - A file that contains a collection of related functions and variables
 - Needs to be added or “imported” to the main program explicitly by using the ***import*** command
 - Syntax: **import module_name**
 - Creates a module object with the module_name
 - Can print the module_name to get information about the module object
 - To access the parts of module, use ***dot operation***, i.e. you specify the name with a dot
 - Syntax for dot operation: **module_name.function_name(arguments)**
 **This calls the function_name from the module_name class **
 - For e.g., `math.log10(avg)` #accessing the function log10 of math module
 `math.pi` #accessing the variable pi of math module

LIBRARY FUNCTIONS

- The modules as explained earlier are “commonly” known as ***Libraries***
- Importing the entire module/library (for e.g. **import math**) may not be necessary, because we might be using only few functions and/or variables of the module
- Use **from** and **import** keywords to access some of the elements present in the processor
- For e.g. **from math import pi**
 from math import sqrt
- This will only bring the **pi** variable and **sqrt** function into the current program

IMPORTING WITH FROM

- When an entire module is imported, the specific functions and variables are addressed by the library name and a dot operator followed by the function/variable, for e.g.

```
import math
```

```
radians = degrees/360.0 * 2 * math.pi
```

- In case of importing some of them, they can be directly used in the instructions, for e.g.

```
from math import pi
```

```
radians = degrees/360.0 * 2 * pi
```

IMPORTING WITH FROM

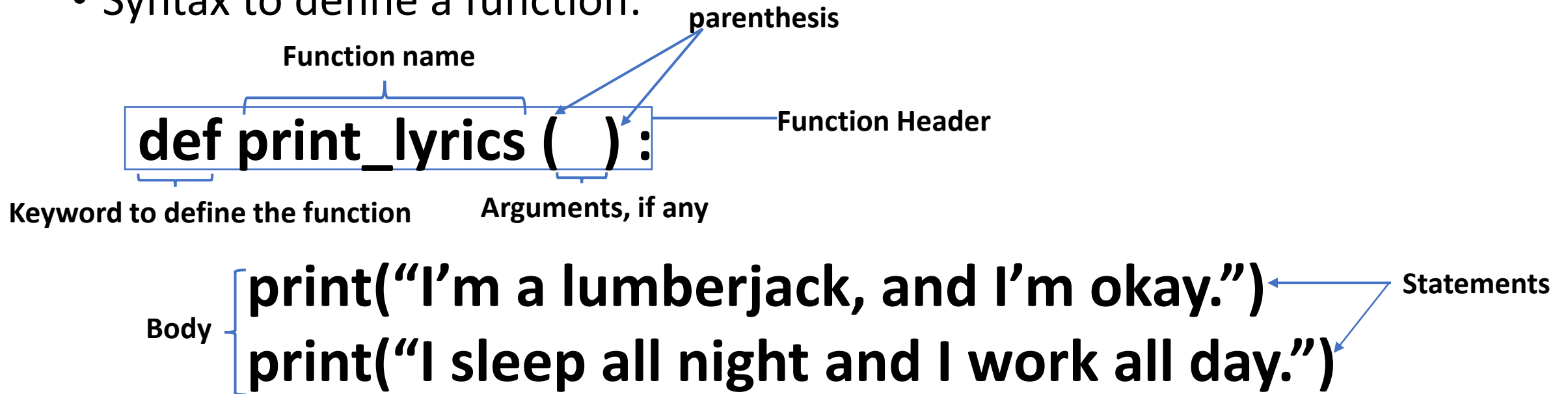
- Advantages:
 - No naming techniques like **dot operator** needed to use the functions/variables
 - Saves memory space as only some functions and/or variables are copied
- Disadvantages:
 - Names of variables and/or functions may conflict with other library functions
 - Naming of local variables and functions must be checked to avoid error

USER-DEFINED FUNCTIONS

- Defined by the programmer to specify his/her set of instructions and requirements
- Must specify:
 - Function Definition
 - Python keyword: ***def***
 - Function name
 - Argument list, if any
 - Body of the function
 - Function Call
 - Invoking by using the format of Function definition

FUNCTION DEFINITION

- Syntax to define a function:



FUNCTION DEFINITION REQUIREMENTS

- **def:**
 - Python keyword to indicate that it's the function's definition
- **Function name:**
 - Same rules as naming variables
 - Letters, numbers and some punctuation marks are permitted
 - First character cannot be a number
 - Cannot use keywords as names
- **Parenthesis:**
 - Follows the function name to hold the arguments of the function
 - If empty, it means no arguments

FUNCTION DEFINITION REQUIREMENTS

- **Arguments:**
 - Zero or more
 - Value(s) that is passed in a function
- **Function Header:**
 - First line of the function
 - Must end with a colon
- **Body:**
 - Rest of the statements under function header
 - Must be indented
 - Can have as many statements

FUNCTION DEFINITION REQUIREMENTS

- **Statement(s):**

- At least one statement is required
- Can represent any operation, for e.g. print, input, arithmetic operations etc.
- May use variables in statements
- May use any form of direct value, for e.g. integer, float, string, etc.
- If using string data, it must be enclosed in quotation marks (double or single)
- However, if string data is already using single quotation like in case of apostrophes, then it MUST be enclosed in double quotation marks

FUNCTION CALL

- Once the function has been properly defined, you can invoke a function to execute it
- Can be done as an independent statement
- Can also be called within another function

FUNCTION CALL EXAMPLE

1. `def print_lyrics( ):`
2. `print ("I'm a lumberjack, and I'm okay.")`
3. `print("I sleep all night and I work all day.")`

4. `def repeat_lyrics( ):`
5. `print_lyrics( )` **#Function call within another function**
6. `print_lyrics( )`

7. `repeat_lyrics( )` **#Function call as an independent statement**

FUNCTIONS

- Function headers are executed just like any other statement, but the result is *function objects*
- The statements are not executed until the function has been called by any part of the program
- Function must be defined before it can be invoked/called
- For the given code in the previous slide, answer the following:
 1. If the statement # 7 is written at the top of the program (i.e. before statement # 1, what will happen?
 2. If definition of `print_lyrics( )` is interchanged with the definition of `repeat_lyrics( )`, what will be the result of the program?

FLOW OF EXECUTION

- Defines the order in which the instructions/statements are executed in the program
- Execution begins with the very first statement
- Statements are executed one at a time, from top to bottom
- Function definitions do NOT alter the sequential execution, as the headers are executed to define function objects, and the body of the function is skipped
- Only in case of function call, the sequential execution takes a detour, such that it goes to the place where function object was defined, it executes the body of the function, and after finishing, it returns back to where the function was called from

PARAMETERS AND ARGUMENTS

- The value that you pass in a function is called it's *argument*
- These arguments are assigned to variables called *parameters*, as defined by the function header
- For e.g.

```
def print_twice(bruce):  
    print (bruce)  
    print (bruce)  
  
print_twice("Hello World!")
```
- **bruce** is a variable that holds all incoming values of the function, hence called *parameter*
- **Hello World!** is the value being transferred to the function, hence called *argument*

PARAMETERS AND ARGUMENTS

- Syntax rules same as variables and/or function names
- Every argument is ***local*** inside the function, i.e. it only exists for the function while it's running.
- Variables defined inside the function body are also ***local*** to the function
- Both argument values, as well as variables are discarded when the function terminates abruptly or normally

RETURN VALUES

- When a function call sends a result back to where the function was called from, that result is known as its ***return value***
- If a resultant value is expected, the caller (statement that called the function) should handle the value
- For e.g. **math.sqrt(5)**
 - A built-in function that is supposed to calculate the square root of the argument given, which in this case is 5, and whatever the square root is, is returned back.
 - If the function call is written as above, even though it will calculate the square root but it will not be able to keep track of it for future references
 - So the return value is either printed on screen or saved in another variable, else it will be useless

VOID FUNCTION

- Functions that do NOT return a value
 - Other functions that return a value are called *fruitful functions*
- They perform all the tasks just like other functions do but they do not return any value, for e.g. **print("Bing")** will simply print the word "Bing", and as you do not need any return messages here, you can write this function as void function
- No assignment needed
- Even if we do assign it to check, we will have a return value of ***None***
- ***None*** is a keyword used in Python to represent null values. It is not the same as literary word "None"
- It has it's own type i.e. ***NoneType***

MAIN FUNCTION

- The top most function of the program, i.e. the starting point of execution
- The program must be written in the main() function in order to run it
- Execution is still sequential from top to bottom, but it must start where main has been defined
- Syntax:

```
def main( ):
    print("Hello World!")
if __name__ == "__main__":
    main()
```

- If no main function is defined, it will print the entire family of function objects